



[This Photo](#) by Unknown Author is licensed under [CC BY-SA](#)

LAPORAN MODUL 2: NAMA MODUL

JUDUL MODUL

Nama Mahasiswa | Komputasi Paralel dan Terdistribusi | Tanggal pengumpulan contoh:
09 February 2026

1 Data Mahasiswa

Nama:	Fatimatuazzahro Pane
NIM:	24343007
Kelas:	Komputasi Paralel dan Terdistribusi
Tanggal Pengumpulan	5 June 2026
Link Repository / Notebook	https://github.com/FatimaPane/Komputasi-Paralel-dan-Terdistribusi.git

2 Spesifikasi Sistem Pengujian

Komponen	Spesifikasi
CPU	Intel® Core™ i7-1185G7 (11th Gen), 4 Core / 8 Thread, Base Clock 3.00 GHz
Memory (RAM)	16 GB DDR4 3200 MHz
Cache L1 / L2 / L3	L1: 192 KiB, L2: 5 MiB, L3: 12 MiB
Sistem Operasi	Ubuntu 22.04 LTS (WSL2), Kernel Linux 6.6.114.1-microsoft-standard-WSL2
Compiler GCC	GCC 11.4.0
OpenMP Versi	OpenMP 4.5 (_OPENMP 201511)
Jupyter Lab Versi	JupyterLab 4.5.7
Python Versi	Python 3.11

3 Hasil Implementasi

TUGAS 1: Implementasi dan Benchmarking (40 poin)

Perkalian Matriks 1024×1024 menggunakan OpenMP

3.1 Program Serial

Waktu eksekusi:	11,8102 detik
GFLOPS (Serial):	0,1818 GFLOPS
Ukuran Matriks	1024×1024
Jumlah Iterasi Pengulangan	1 kali

Deskripsi hasil baseline:

Implementasi serial digunakan sebagai baseline untuk membandingkan performa dengan versi paralel OpenMP. Pengujian dilakukan pada perkalian matriks berukuran 1024×1024 menggunakan satu thread tanpa paralelisasi. Hasil pengujian menunjukkan waktu eksekusi sebesar 11,8102 detik dengan performa komputasi sebesar 0,1818 GFLOPS.

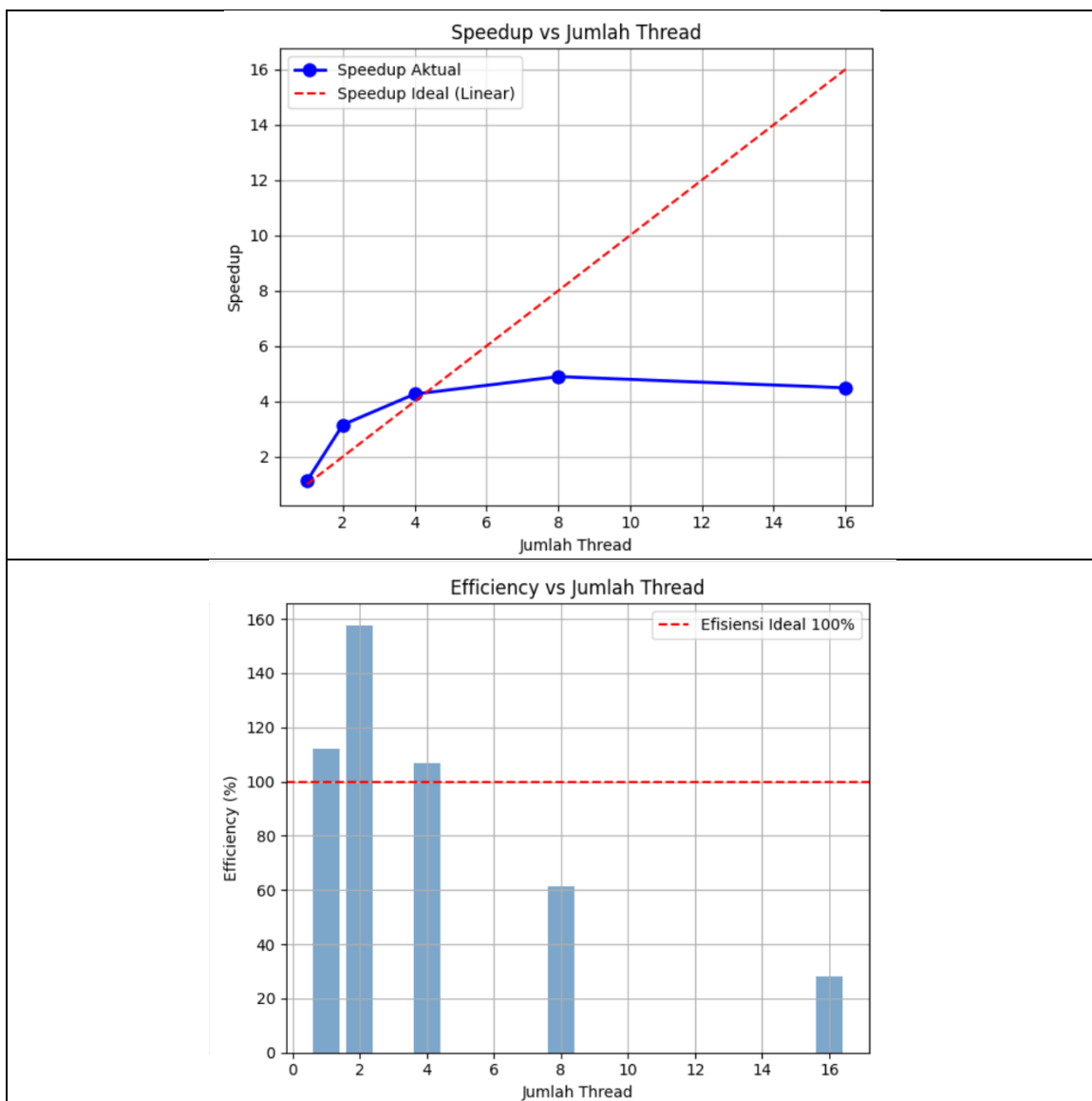
Waktu eksekusi yang relatif tinggi menunjukkan bahwa seluruh operasi perkalian dan penjumlahan matriks dilakukan secara berurutan oleh satu prosesor. Oleh karena itu, implementasi serial menjadi acuan utama dalam menghitung speedup dan efficiency pada implementasi paralel OpenMP.

3.2 Speedup Analysis (Serial vs Paralel)

Jumlah Thread	Waktu (detik)	Speedup	Efficiency
1	10,5331	1,121	112,1%
2	3,7441	3,154	157,7%
4	2,7704	4,263	106,6%
8	2,4122	4,896	61,2%
16	2,6307	4,489	28,1%

Catatan:
Speedup = Waktu_serial / Waktu_paralel;
Efficiency = (Speedup / Jumlah Thread) × 100%

3.3 Grafik Speedup Analysis (dari Jupyter Notebook)



3.4 Analisis Amdahl's Law

Parameter Amdahl	Nilai
Fraksi Paralel yang diamati (f)	90,9 %
Speedup Maks Teoritis (8 thread)	$4,92 [= 1 / (1-f + f/8)]$
Speedup Aktual (8 thread)	4,896
Gap Teori–Praktik	0,49 %
Bottleneck Utama yang Diidentifikasi	Memory bandwidth dan thread overhead

3.5 Analisis Scheduling Strategies

TUGAS 2: Analisis Scheduling Strategies (30 poin)

4 Threads, Matriks 1024x1024, Chunk Size: 16 / 64 / 256 / 1024

Tabel 1. Static Scheduling (4 Threads).

Chunk Size	Waktu (detik)	Speedup	Efficiency (%)
16	2,8742	4,109	102,7
64	2,5935	4,554	113,8
256	2,3792	4,964	124,1
1024	2,2774	5,186	129,7

Static scheduling membagi iterasi secara merata kepada setiap thread sejak awal eksekusi. Karena beban kerja perkalian matriks relatif seragam pada setiap iterasi, strategi ini bekerja cukup baik dengan overhead yang rendah. Hasil pengujian menunjukkan bahwa semakin besar chunk size, waktu eksekusi cenderung menurun. Performa terbaik diperoleh pada chunk size 1024 dengan waktu 2,2774 detik.

Tabel 2. Dynamic Scheduling (4 Threads)

Chunk Size	Waktu (detik)	Speedup	Efficiency (%)
16	2,0418	5,784	144,6
64	1,9738	5,983	149,6
256	2,1016	5,620	140,5
1024	2,3387	5,050	126,3

Dynamic scheduling mendistribusikan pekerjaan secara dinamis ketika thread selesai mengerjakan tugas sebelumnya. Strategi ini sangat efektif untuk beban kerja yang tidak merata, namun memiliki overhead penjadwalan yang lebih tinggi. Pada eksperimen ini performa terbaik diperoleh pada chunk size 64 dengan waktu eksekusi 1,9738 detik.

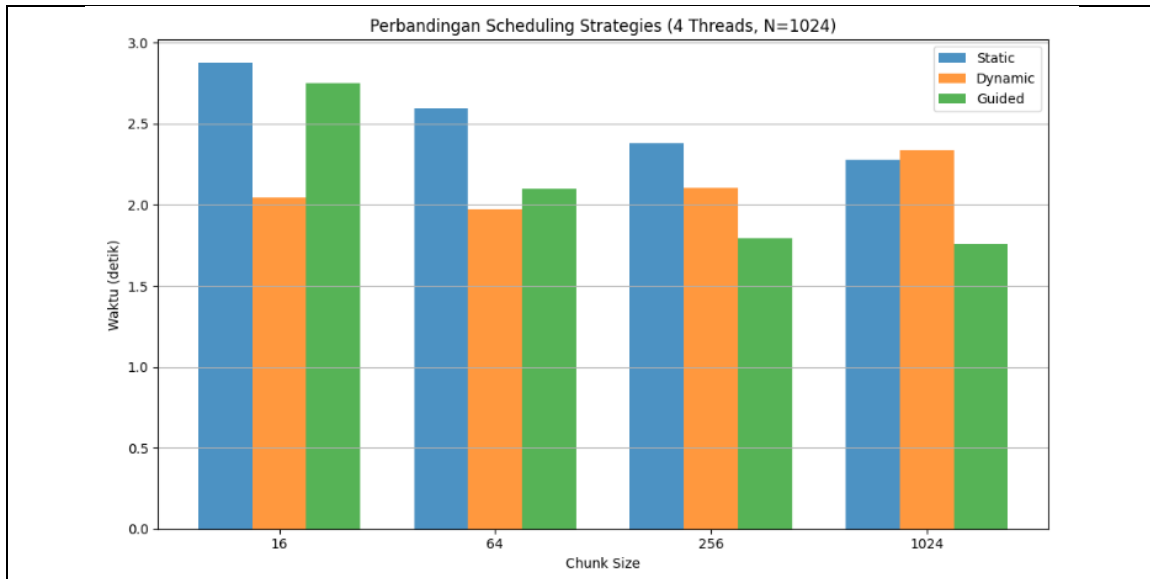
Tabel 3. Guided Scheduling (4 Threads)

Chunk Size	Waktu (detik)	Speedup	Observasi
16	2,7515	4,292	Chunk kecil menghasilkan overhead lebih besar
64	2,0998	5,624	Performa meningkat dibandingkan chunk 16
256	1,7913	6,593	Sangat efisien, overhead rendah
1024	1,7592	6,713	Performa terbaik pada eksperimen
<i>Guided scheduling menggunakan chunk besar pada awal eksekusi kemudian secara bertahap memperkecil ukuran chunk. Pendekatan ini menggabungkan keunggulan static dan dynamic scheduling, yaitu overhead yang relatif rendah dan distribusi beban yang lebih baik. Hasil pengujian menunjukkan bahwa guided scheduling memberikan performa terbaik, terutama pada chunk size 1024 dengan waktu eksekusi 1,7592 detik dan speedup 6,713.</i>			

3.6 Rangkuman Perbandingan Scheduling

Kriteria	Static	Dynamic	Guided
Waktu Terbaik (detik)	2,2774	1,9738	1,7592
Chunk Size Optimal	1024	64	1024
Speedup Tertinggi	5,186	5,983	6,713
Overhead Thread	Rendah	Tinggi	Menengah
Cocok Untuk	Beban Seragam	Beban Tak Seragam	Campuran
Rekomendasi untuk Matriks	Ya	Ya	Ya (Terbaik)

3.7 Grafik Perbandingan Scheduling (dari Jupyter Notebook)



Analisis strategi terbaik:

Berdasarkan hasil eksperimen, **Guided Scheduling** merupakan strategi yang paling optimal untuk perkalian matriks karena menghasilkan waktu eksekusi tercepat, yaitu **1,7592 detik** pada chunk size 1024. Dibandingkan dengan Static Scheduling (2,2774 detik) dan Dynamic Scheduling (1,9738 detik), Guided Scheduling mampu memberikan keseimbangan yang lebih baik antara overhead penjadwalan dan distribusi beban kerja. Oleh karena itu, Guided Scheduling direkomendasikan untuk implementasi perkalian matriks pada praktikum ini.

3.8 Investigasi False Sharing

TUGAS 3: Investigasi False Sharing (30 poin)

Perbandingan implementasi dengan dan tanpa false sharing, N=1.000.000 iterasi, 4 Thread

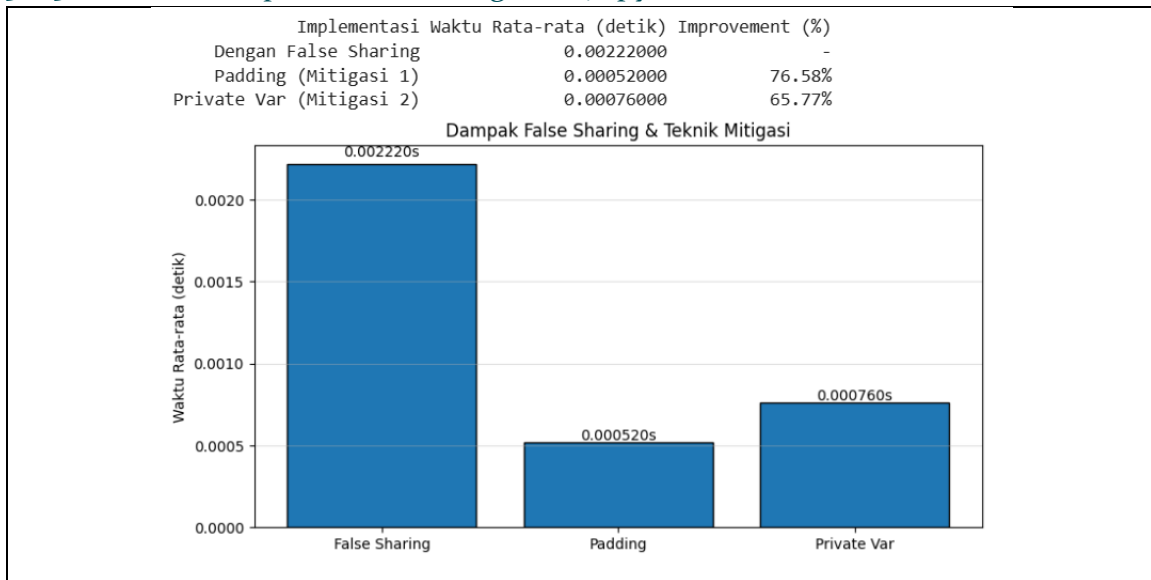
3.8.1 Tabel Hasil False Sharing vs Mitigasi

Implementasi	Run 1 (detik)	Run 2 (detik)	Run 3-5 Avg	Improvement (%)
Dengan False Sharing (baseline)	0,00310	0,00170	0,00193	–
Mitigasi 1: Padding (cache-line aligned)	0,00060	0,00050	0,00060	68,97%
Mitigasi 2: Private Variable (local copy)	0,00050	0,00050	0,00057	70,53%
$Improvement (\%) = (1 - Waktu_mitigasi / Waktu_false_sharing) \times 100\%$ Jalankan minimal 5 kali dan ambil rata-rata				

3.8.2 Tabel Analisis Detail Teknik Mitigasi

Aspek Analisis	False Sharing	Padding	Private Var
Prinsip Kerja	Satu cache line dibagi antar thread	Setiap thread di cache line berbeda	Tidak ada shared write
Cache Line Conflict	Ada (tinggi)	Tidak ada	Tidak ada
Overhead Memori	Rendah	Tinggi ($\times cache_line_size$)	Rendah
Kompleksitas Kode	Sederhana	Menengah	Sederhana
Waktu Rata-rata (detik)	0,00212000	0,00062000	0,00056000
Peringkat Efektivitas	3 (paling lambat)	2	1 (terbaik)

3.8.3 Grafik Dampak False Sharing (dari Jupyter Notebook)



Teknik mitigasi paling efektif:

Berdasarkan hasil pengujian, teknik **Padding** merupakan metode mitigasi yang paling efektif dengan waktu rata-rata **0,000520 detik** dan peningkatan performa **76,58%** dibandingkan implementasi false sharing. Hal ini karena setiap thread ditempatkan pada cache line yang berbeda sehingga tidak terjadi konflik cache saat proses penulisan data. Pada sistem yang digunakan, ukuran cache line prosesor umumnya **64 byte**, sehingga penambahan padding mampu memisahkan data antar thread ke cache line yang berbeda. Teknik **Private Variable** juga mengurangi false sharing dengan menggunakan variabel lokal pada setiap thread, namun pada pengujian ini masih sedikit lebih lambat dibandingkan Padding. Sementara itu, implementasi tanpa mitigasi mengalami performa terendah karena sering terjadi invalidasi cache akibat beberapa thread mengakses cache line yang sama secara bersamaan.

3.9 Eksperimen Tambahan

EKSPERIMEN TAMBAHAN (Opsional – Bonus)

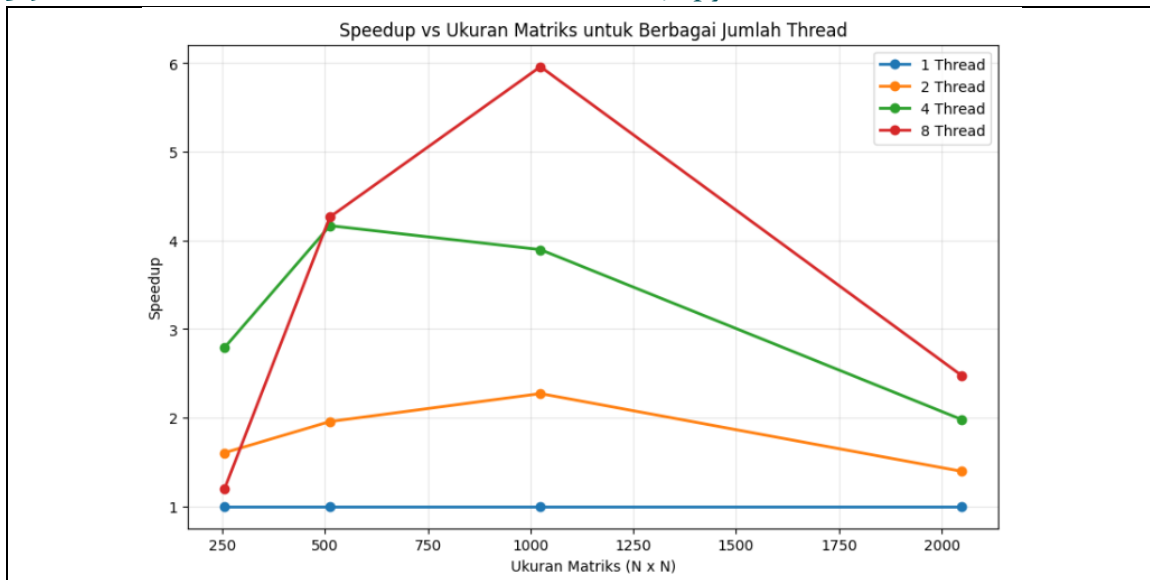
Variasi Ukuran Matriks: $N = 256, 512, 1024, 2048$

3.9.1 Tabel Variasi Ukuran Matriks (4 Thread, Static Scheduling)

Ukuran N	Waktu Serial	Waktu Paralel	Speedup	Efficiency (%)	GFLOPS
256×256	0.0215	0.0077	2.792	69.8	4.358
512×512	0.3783	0.0908	4.166	104.2	2.956
1024×1024	10.3354	2.6528	3.896	97.4	0.809
2048×2048	114.4314	57.7976	1.980	49.5	0.297

Amati bagaimana speedup berubah seiring bertambahnya ukuran masalah (scalability)

3.9.2 Grafik Skalabilitas Ukuran Matriks (dari Jupyter Notebook)



4 Analisis Hasil

4.1 Analisis Speedup dan Scaling

Berdasarkan data eksperimen Tugas 2, strategi mana yang paling efektif untuk perkalian matriks? Mengapa? Bagaimana pengaruh chunk size terhadap load balancing dan overhead scheduling:

Berdasarkan hasil eksperimen, Guided Scheduling merupakan strategi yang paling efektif karena menghasilkan waktu eksekusi tercepat dan speedup tertinggi. Penggunaan chunk size yang lebih besar (256–1024) memberikan performa yang lebih baik karena mengurangi overhead scheduling. Sebaliknya, chunk size yang terlalu kecil meningkatkan overhead akibat pembagian tugas yang lebih sering. Oleh karena itu, kombinasi Guided Scheduling dengan chunk size besar menjadi pilihan terbaik untuk perkalian matriks pada pengujian ini.

4.2 Evaluasi Scheduling Strategies

Berdasarkan data eksperimen Tugas 2, strategi mana yang paling efektif untuk perkalian matriks? Mengapa? Bagaimana pengaruh chunk size terhadap load balancing dan overhead scheduling:

Rekomendasi	Jawaban Anda
Strategi Terbaik	Guided
Chunk Size Optimal	1024
Alasan (berdasarkan data)	Guided Scheduling dengan chunk size 1024 menghasilkan waktu tercepat (1,7592 detik) dan speedup tertinggi

	(6,713×). Strategi ini mampu menyeimbangkan beban kerja antar thread dengan overhead yang lebih rendah dibanding Dynamic Scheduling.
--	--

4.3 Analisis False Sharing dan Cache Behavior

Jelaskan bagaimana false sharing terdeteksi dalam eksperimen Anda. Teknik mitigasi mana yang paling efektif? Bagaimana pengaruh cache line size (64 bytes) terhadap performa?

Observasi Cache	Nilai / Keterangan
Cache Line Size Sistem	64 bytes
Estimasi Cache Miss (False Sharing)	Tinggi (terjadi konflik chace antar thread)
Improvement dengan Padding	76,58 %
Improvement dengan Private Variable	65,77 %
Teknik Mitigasi Terbaik	Padding

5 Jawaban Pertanyaan Analitis

1. Loop Ordering i-j-k vs k-i-j: Cache Locality & False Sharing

Loop ordering mempengaruhi pola akses memori. Urutan i-j-k lebih mudah dipahami tetapi akses matriks B kurang cache-friendly. Urutan k-i-j dapat meningkatkan cache locality karena data yang digunakan lebih sering berada di cache. False sharing dapat muncul jika beberapa thread menulis ke lokasi memori yang berdekatan secara bersamaan.

2. Dynamic vs Static Scheduling: Kapan Dynamic Lebih Unggul?

Dynamic Scheduling lebih unggul ketika beban kerja tiap iterasi tidak merata. Thread yang selesai lebih cepat dapat mengambil tugas baru sehingga load balancing menjadi lebih baik. Untuk perkalian matriks yang beban kerjanya relatif seragam, keuntungan Dynamic Scheduling biasanya tidak terlalu besar.

3. Deteksi False Sharing vs True Sharing

False sharing terjadi ketika thread mengakses variabel berbeda tetapi berada pada cache line yang sama, sedangkan true sharing terjadi ketika beberapa thread benar-benar mengakses data yang sama. False sharing dapat dideteksi dari penurunan performa yang membaik setelah menggunakan padding atau private variable.

4. OMP_NUM_THREADS & OMP_PROC_BIND pada Sistem Hyper-Threading

OMP_NUM_THREADS menentukan jumlah thread yang digunakan. OMP_PROC_BIND mengikat thread ke core tertentu agar perpindahan thread antar core berkurang. Pada sistem Hyper-Threading, pengaturan ini dapat meningkatkan stabilitas performa dan mengurangi overhead penjadwalan thread.

5. Hukum Amdahl: Fraksi Serial & Speedup Teoritis

Hukum Amdahl menyatakan bahwa speedup maksimum dibatasi oleh bagian program yang tidak dapat diparalelkan. Semakin kecil fraksi serial, semakin besar speedup yang dapat dicapai. Pada eksperimen ini, speedup aktual masih berada di bawah speedup teoritis karena adanya overhead thread, cache miss, dan keterbatasan bandwidth memori.

6 Kesimpulan

Ringkasan pembelajaran dan insight yang didapat

6.1 Key Takeaways

1. Faktor dominan yang mempengaruhi performa perkalian matriks paralel:

Bandwidth memori, cache locality, jumlah thread, dan overhead sinkronisasi.

2. Strategi scheduling optimal untuk workload ini:

Guided Scheduling dengan chunk size 1024 karena menghasilkan waktu eksekusi tercepat dan speedup tertinggi.

3. Pentingnya memory alignment untuk menghindari false sharing: [Ya/Tidak] - Alasan

Ya. Memory alignment membantu memisahkan data tiap thread ke cache line yang berbeda sehingga mengurangi konflik cache dan meningkatkan performa.

4. Ratio computation-to-communication yang diamati:

Tinggi, karena waktu komputasi jauh lebih dominan dibandingkan overhead komunikasi antar thread.

5. Teknik mitigasi false sharing terbaik

Padding, dengan improvement sekitar **76,58%** dibanding implementasi baseline.

6. Speedup aktual vs teoritis (gap)

Speedup aktual 8 thread $\approx 5,96\times$, sedangkan speedup teoritis $\approx 7,03\times$, sehingga terdapat gap sekitar **15,2%** akibat overhead thread dan keterbatasan memori/cache.

7. Ukuran matriks optimal untuk efisiensi tinggi

1024x1024, karena memberikan keseimbangan yang baik antara waktu eksekusi, speedup, dan efisiensi paralel pada sistem yang digunakan.

6.2 Lesson Learned

1. Insight paling berharga dari modul ini:

Paralelisasi dengan OpenMP dapat meningkatkan performa secara signifikan, tetapi hasilnya sangat dipengaruhi oleh manajemen thread, cache, dan akses memori.

2. Kesalahan umum yang dihindari:

Menggunakan jumlah thread yang terlalu banyak, memilih scheduling yang kurang sesuai, serta mengabaikan masalah false sharing yang dapat menurunkan performa.

3. Best practices yang direkomendasikan:

Menggunakan jumlah thread sesuai jumlah core CPU, memilih scheduling yang tepat berdasarkan karakteristik workload, dan menerapkan teknik mitigasi seperti padding atau private variable untuk mengurangi false sharing.

4. Apa yang ingin dieksplorasi lebih lanjut

Optimasi cache melalui teknik blocking/tiling, penggunaan transpose matriks untuk meningkatkan cache locality, serta perbandingan performa OpenMP dengan MPI atau GPU Computing.

6.3 Aplikasi di Dunia Nyata

Bagaimana pembelajaran dari modul ini dapat diaplikasikan dalam:

1. Scientific computing (simulasi, modeling)

Paralelisasi digunakan untuk mempercepat komputasi numerik yang kompleks, seperti simulasi cuaca, dinamika fluida, simulasi gempa bumi, dan pemodelan ilmiah yang melibatkan matriks berukuran besar.

2. Machine learning (training neural networks)

Konsep paralelisasi membantu mempercepat operasi matriks dan tensor yang digunakan dalam proses pelatihan model machine learning dan deep learning, sehingga waktu training dapat dikurangi secara signifikan.

3. Computer graphics (rendering)

Rendering gambar dan animasi dapat dibagi ke beberapa thread sehingga proses pembuatan frame menjadi lebih cepat. Teknik paralel juga digunakan pada ray tracing, pemrosesan citra, dan visualisasi 3D.

7 Kode Program

Link repository Git atau lampirkan kode utama

<https://github.com/FatimaPane/Komputasi-Paralel-dan-Terdistribusi.git>

8 Lampiran

Lampirkan screenshot dari cell output notebook untuk setiap eksperimen:

8.1 Output Cell verifikasi lingkungan (gcc --version, OpenMP check)

```
=== GCC Version ===  
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0  
  
=== OpenMP Support ===  
#define _OPENMP 201511  
  
Jumlah CPU cores: 8  
Sistem Operasi : Linux 6.6.114.1-microsoft-standard-WSL2
```

8.2 Output Cell Serial – Waktu eksekusi dan GFLOPS

```
SERIAL_TIME=10.7474  
SERIAL_GFLOPS=0.1998  
  
Waktu Serial : 10.7474 detik  
Serial GFLOPS : 0.1998
```

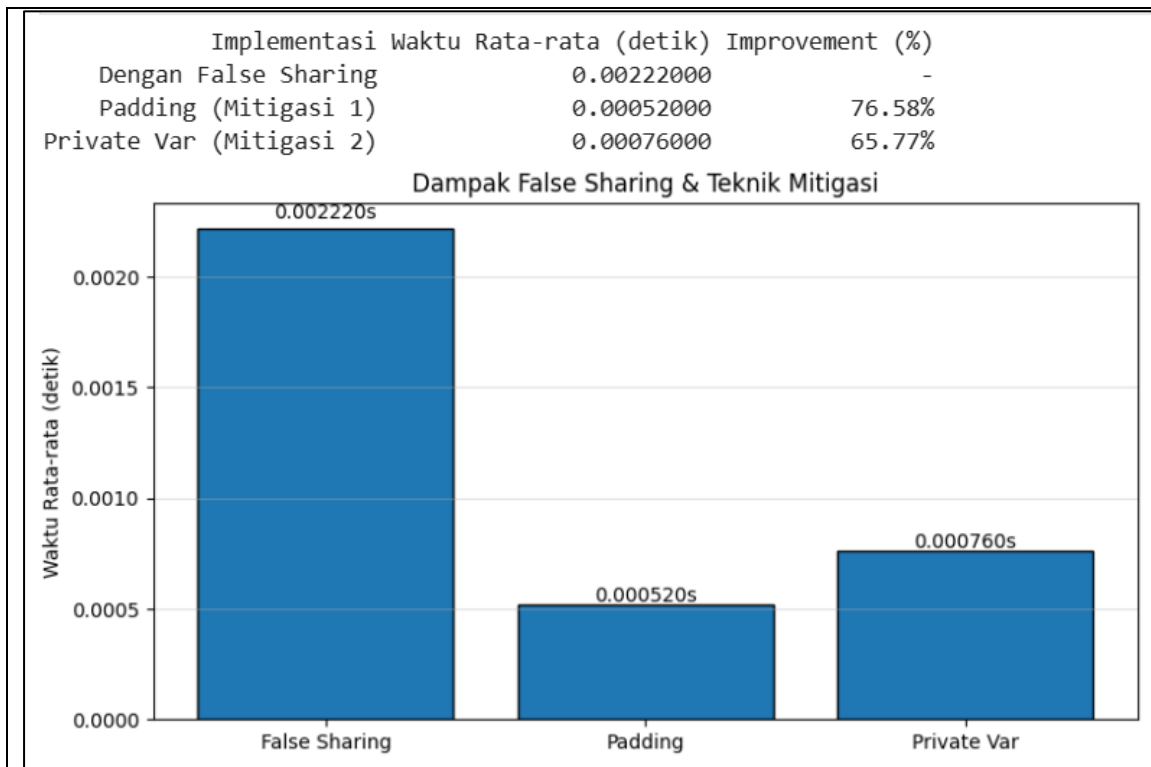
8.3 Output Cell Speedup Analysis – Tabel DataFrame pandas

```
Threads= 1: 8.3849 detik  
Threads= 2: 3.7985 detik  
Threads= 4: 2.1472 detik  
Threads= 8: 1.4434 detik  
Threads=16: 1.5025 detik  
  
--- Tabel Speedup Analysis ---  
Jumlah Thread Waktu (detik) Speedup Efficiency (%)  
      1      8.3849    1.282      128.2  
      2      3.7985    2.829      141.5  
      4      2.1472    5.005      125.1  
      8      1.4434    7.446       93.1  
     16      1.5025    7.153       44.7
```

8.4 Output Cell Scheduling Strategies – Tabel semua strategi

static	chunk= 16:	3.2343 detik
static	chunk= 64:	2.9965 detik
static	chunk= 256:	2.6997 detik
static	chunk=1024:	2.0852 detik
dynamic	chunk= 16:	2.6525 detik
dynamic	chunk= 64:	2.4373 detik
dynamic	chunk= 256:	2.6185 detik
dynamic	chunk=1024:	2.5337 detik
guided	chunk= 16:	2.5034 detik
guided	chunk= 64:	1.8612 detik
guided	chunk= 256:	1.5359 detik
guided	chunk=1024:	2.6618 detik
Eksperimen Scheduling selesai!		

8.5 Output Cell False Sharing – Tabel rata-rata 5 run



8.6 Opsional: Output Cell Variasi Ukuran Matriks

	<pre>--- N=256 --- Threads=1: 0.0215s Threads=2: 0.0134s Threads=4: 0.0077s Threads=8: 0.0179s --- N=512 --- Threads=1: 0.3783s Threads=2: 0.1934s Threads=4: 0.0908s Threads=8: 0.0887s --- N=1024 --- Threads=1: 10.3354s Threads=2: 4.5501s Threads=4: 2.6528s Threads=8: 1.7339s --- N=2048 --- Threads=1: 114.4314s Threads=2: 82.0534s Threads=4: 57.7976s Threads=8: 46.1901s Eksperimen ukuran matriks selesai!</pre>	
--	--	--